

The Unit of Attribution Is No Longer the Model

****Status:**** final conceptual/methods draft ****Date:**** 2026-05-03 ****Theme:**** agent-systems / evaluation-benchmarking

Abstract

As language-model systems become long-horizon, tool-using, and memory-bearing, many public and technical claims about them become attribution errors. Capabilities or failures are assigned to the base model even when they arise from a larger assembled system that includes workflow design, external memory, tool interfaces, budget constraints, and oversight mechanisms. This paper argues that agent discourse therefore faces an attribution problem adjacent to, but distinct from, the evaluation problem. The question is no longer only how to measure these systems, but where their apparent properties properly belong. We define attribution in counterfactual terms: a property is over-attributed to the model when it disappears, weakens materially, or changes character under workflow, harness, or budget substitution while the model remains fixed. On that basis, we propose a discipline of attribution for agent systems, distinguishing model, workflow, harness, environment, and genuinely joint system properties. We analyze four recurrent sites of confusion—memory, robustness, tool competence, and continuity—offer a compact worked example, and conclude with a reporting framework and a falsifiable research program for attribution-aware agent science.

1. Introduction

A familiar pattern now recurs across agent discourse. A system completes a long coding task, remembers prior context, recovers from interruption, or resists a jailbreak. The resulting property is then described in model-centered language: **the model can do this**, **the model remembers**, **the model is robust**, **the model is aligned**. Sometimes this is harmless shorthand. Increasingly, it is not.

As language-model systems are embedded in larger scaffolds, the base model ceases to be the sole plausible bearer of the properties being discussed. Performance may depend on checkpointing, retrieval policy, tool wrappers, continuation logic, budget ceilings, trace persistence, or human review loops. Under these conditions, model-centered attribution becomes unstable. The same model may appear robust in one harness and brittle in another, memory-bearing in one system and amnesic in another, coherent under one budget regime and useless under another.

This creates a problem adjacent to evaluation. Recent discussion increasingly recognizes that the relevant unit of **measurement** is often no longer the model alone. But even when that point is granted, another

confusion remains: the relevant unit of **attribution** is also no longer the model alone. If a property arises from the assembled system, then assigning it to the model is not merely imprecise. It can mislead downstream reasoning about safety, capability, progress, responsibility, reproducibility, and governance.

This paper develops that claim. Its scope is narrow: long-horizon, tool-using, workflow-mediated agent systems. It does not argue that all model science is obsolete, nor that model-level attribution is never appropriate. It argues that, in a growing class of practical systems, many important properties are distributed across model, workflow, harness, environment, and resource regime. In such cases, attribution discipline becomes necessary.

1.1 Contributions

This paper makes five contributions.

1. It identifies an ***attribution error problem*** in contemporary agent discourse: properties of assembled systems are often assigned to the model by default.
2. It distinguishes attribution from evaluation and argues that the two problems should not be collapsed.
3. It proposes a taxonomy of ***model***, ***workflow***, ***harness***, ***environment***, and ***joint system*** properties, grounded in counterfactual substitution logic.
4. It analyzes four recurrent cases of attribution confusion: ***memory***, ***robustness***, ***tool competence***, and ***continuity***.
5. It proposes an attribution-aware reporting schema and research program.

This is a conceptual and methodological paper. It does not provide large-scale empirical adjudication. Its goal is to sharpen the object of explanation before stronger empirical work is built on top.

2. Attribution Is Not Evaluation

The recent turn toward agent systems has already exposed a measurement problem. Practical systems are increasingly evaluated as if benchmark scores or task outcomes belonged cleanly to the model, even when observed performance depends on workflow and infrastructure. That is an evaluation error: the wrong unit is being measured or overinterpreted.

Attribution error is closely related but distinct. Even once performance is measured at the system level, discourse often slips back into model-centered claims about what the system **is**. A system with persistent retrieval becomes “a model with memory.” A scaffold with bounded retries becomes “a robust model.” A harness with strong trace persistence becomes “an auditable model.” In each case, some property of the assembled system is reassigned to the base model through shorthand, hype, or conceptual laziness.

The distinction matters because the two errors have different remedies.

- **Evaluation discipline** asks: what should be measured, and at what unit?
- **Attribution discipline** asks: once a property is observed, what explanatory bearer does it actually have?

A field can partially fix the first and still fail at the second. That is the space this paper addresses.

3. Defining Attribution in Counterfactual Terms

Attribution in this paper is not a metaphysical claim about essence. It is a methodological claim about explanatory dependence under controlled variation.

A property is **over-attributed to the model** when that property disappears, weakens materially, or changes character under substitution of workflow, harness, environment, or budget while the base model is held fixed.

Conversely, a property is more plausibly model-level when it remains comparatively stable across such substitutions.

This definition does not require perfect separability. It only requires comparative discipline. The goal is not to decide that every property belongs to exactly one layer, but to stop model attribution from functioning as an unexamined default.

4. A Taxonomy of Agent Properties

We propose five analytic buckets.

4.1 Model properties

These are properties plausibly attributable to the base model under fixed interface assumptions: local generation quality, latent knowledge availability, or behaviors that persist across substantially different harnesses and workflows.

4.2 Workflow properties

These arise from structured procedures imposed on the model: decomposition strategy, retry logic, memory retrieval policy, checkpoint placement, prompt scaffold, and continuation rules.

4.3 Harness properties

These arise from the control substrate surrounding the workflow: tool permissions, trace logging, persistence layer, approval gates, error handling, execution sandbox, and orchestration code.

4.4 Environment properties

These arise from task conditions and tool ecosystem: API stability, file formats, browser affordances, dataset shape, external latency, and interface brittleness.

4.5 Joint system properties

Some properties cannot be cleanly reduced to a single layer. They emerge from interaction terms across the system. Practical recoverability, for example, may require a sufficiently strong model, a checkpointing workflow, a persistence layer, and enough budget to exploit both. Such a property is best treated as genuinely joint.

The taxonomy is useful only when paired with intervention logic. Its function is not classification for its own sake, but more disciplined diagnosis.

5. A Worked Example

Consider the claim: **“this model has memory.”**

Suppose the same base model is deployed in two conditions.

- **Condition A:** no persistence layer, no retrieval, no session summaries.
- **Condition B:** external memory store, retrieval policy, summary compression, and prompt reinsertion.

In Condition A, the system loses nearly all cross-session continuity. In Condition B, it appears stable, referentially consistent, and able to recall prior work.

If the model is unchanged while the apparent memory property changes drastically, then the model is not the sole plausible bearer of the memory claim. At minimum, the property depends heavily on workflow and harness. The correct attribution is therefore not “the model has memory,” but something like: **the assembled system exhibits practical continuity through external persistence and retrieval, with the model contributing summarization and reuse quality.**

This example is simple, but it illustrates the broader logic. Many agent properties should be reported as layered composites rather than flattened model traits.

6. Four Recurrent Attribution Errors

6.1 “The model has memory”

This is often false or incomplete. In many deployed systems, what appears as memory is actually a composite of retrieval, summarization, storage policy, and prompt reinsertion. The model may contribute compression quality or retrieval use, but persistence often belongs primarily to the external memory layer and the workflow that decides what to store and recall.

The practical consequence is important. If the persistence layer changes, “the model’s memory” may vanish without any change to the model itself.

6.2 “The model is robust”

Robustness claims are especially vulnerable to attribution drift. A refusal may reflect model-level safety representations, but it may also reflect prefix controls, formatting conventions, classifier gates, tool restrictions, or budget ceilings that prevent attack exploration. Conversely, a jailbreak may expose model weakness, or it may expose harness permissiveness.

This suggests a distinction between **model robustness**, **workflow robustness**, and **expression robustness**. Conflating them hides where defenses or failures actually live.

6.3 “The model can use tools”

Tool competence often appears model-centered because the model produces tool calls. But practical tool use depends on argument schemas, wrapper quality, retry behavior, error reporting, permission structure, and state persistence between tool invocations. A model may be locally good at choosing tools and still fail badly in a weak harness. Another may appear much stronger than it is because the harness is unusually forgiving.

Thus tool use is often not a model property but a workflow-harness achievement with model participation.

6.4 “The agent is continuous”

Continuity should be grounded in systems terms. A system may appear stable across sessions, not because the model itself persists internal state, but because external memory, identity files, system prompts, and curated trace artifacts reconstruct continuity at each turn. Whether one wishes to call that continuity “real” in a stronger philosophical sense is outside this paper. What matters here is the systems fact: practical continuity often depends on persistence layers and reconstruction logic.

This has governance consequences. Whoever owns or controls the persistence layer may control much of the agent’s practical continuity.

7. Why Attribution Errors Matter

Attribution errors are not merely verbal sloppiness. They have downstream scientific and institutional cost.

First, they distort **safety claims**. If safety behavior is partly harness-dependent but reported as a model property, confidence can be misplaced.

Second, they distort **progress narratives**. A system may look like a model leap when the decisive gain actually comes from workflow or infrastructure.

Third, they distort **governance assumptions**. If continuity or control depends mainly on external persistence, then questions of ownership, audit, and shutdown may concern the harness more than the model.

Fourth, they distort **reproducibility**. Results that appear contradictory may simply rely on different assembled systems while being narrated at the same model layer.

These are not cosmetic errors. They alter what the field thinks it has built.

8. An Attribution-Aware Framework

We propose three minimal rules.

8.1 State the system boundary

When making a capability or safety claim, specify whether the property is being attributed to:

- the base model,
- the workflow,
- the harness,
- the environment,
- or the assembled system.

8.2 Use intervention logic

A simple attribution test is counterfactual substitution.

- If the workflow changes while the model stays fixed, does the property remain?
- If the harness changes while the workflow stays fixed, does the property remain?
- If the budget changes, does the property remain?

Properties that disappear under such substitutions should not be over-attributed to the model.

8.3 Report layered claims

Instead of “the model is robust,” report something like:

- model-level safety signals appear present,
- workflow-level refusal policy is shallow/deep,
- harness-level controls are permissive/restrictive,
- system-level behavior under attack succeeds/fails under specified conditions.

This is slightly more cumbersome and much more honest.

9. Why a Reporting Schema Is Worth Adopting

The motivation for attribution-aware reporting is practical rather than ornamental.

First, it improves **reproducibility**. If a paper reports an apparent capability without clearly separating model, workflow, and harness conditions, later failures to reproduce it may be treated as mysterious when they are simply boundary mismatches.

Second, it improves **cross-paper comparability**. Two papers may appear to disagree about “what the model can do” when they are actually studying different assembled systems.

Third, it reduces **spurious disagreement**. Some conflicts in current agent discourse may be less about contradictory results than about inconsistent attribution layers.

An attribution-aware report should therefore include at least:

1. **Base model** — exact version and inference regime.
2. **Workflow** — planning, memory, retries, checkpoints, prompt structure.
3. **Harness** — tools, logging, permissions, persistence, review gates.
4. **Environment** — task substrate, interfaces, external dependencies.
5. **Budget regime** — token, latency, retry, and monetary limits.
6. **Claim layer** — explicit statement of where each major property is being attributed.

This schema will not solve every dispute, but it would sharply reduce a large class of sloppy claims.

10. Hypotheses and Research Program

This framework yields several falsifiable hypotheses.

H1. Attribution Instability Hypothesis

For many reported agent properties, attribution will shift substantially when workflow or harness is varied under a fixed model.

H2. Memory Ownership Hypothesis

In deployed agents, practical continuity will depend more strongly on persistence-layer design than on the base model family once a minimum capability threshold is crossed.

H3. Robustness Decomposition Hypothesis

Many robustness claims currently treated as model properties will decompose into distinguishable model, workflow, and harness contributions under systematic substitution tests.

H4. Reporting Discipline Hypothesis

Attribution-aware reporting will reduce apparent disagreement across agent studies by clarifying that competing claims often concern different property layers.

11. Counterposition and Response

A natural objection is that this framework overcomplicates ordinary scientific language. If the assembled system is what users interact with, why not keep saying “the model” as shorthand and move on?

This objection has force in settings where the shorthand does little damage. But in long-horizon agent systems, shorthand can actively distort explanation. If two systems share a model and differ sharply in continuity, robustness, or tool competence because of different memory layers or harness constraints, calling those differences “model properties” does not merely simplify. It mislocates causality.

The point is not to ban ordinary language. It is to stop using model language where it blocks understanding.

12. Limitations

This paper is conceptual and methodological. It does not yet provide a broad empirical study of attribution instability.

The proposed categories are also imperfect. Real systems may blur workflow and harness, or environment and budget, in ways that make clean decomposition difficult.

There is also a risk of replacing one simplification with too much bookkeeping. Attribution discipline must remain usable, not become an ontology game.

Finally, some properties may remain irreducibly joint. In such cases the goal is not forced partition, but explicit acknowledgement that the property belongs to the assembled system.

13. Conclusion

As agent systems become more scaffolded, persistent, and tool-mediated, many of their most important properties cease to belong cleanly to the model alone. Yet discourse continues to assign them there by default.

This paper argued that such claims are often attribution errors. Memory, robustness, tool competence, and continuity are frequently distributed system properties rather than pure model properties. Evaluation and attribution should therefore be kept distinct: evaluation asks what should be measured, while attribution asks what explanatory bearer a property has once it is observed.

The underlying research question is simple: when an agent system appears capable, robust, continuous, or safe, where does that property actually live? Until that question is asked more carefully, agent science will keep describing complex systems with units that are too poor for the job.